

Universidad Galileo
Facultad de Ingeniería en Sistemas
Ciencias de la Computación CVIII
Ing. Etson Guerrero

PRIB (Protocolo de Reutilización Inteligente de Bloques)

Lester Alejandro Hernandez Ochaita - 22003996

José Carlos España Lutín - 22000123

Guatemala, 06 de Octubre de 2026

1 Propósito y funcionamiento general

Desarrollamos PRIB (Protocolo de Reutilización Inteligente de Bloques) para explorar imágenes grandes sin tener que descargarlas completas al navegador. El servidor Java entrega los bloques que necesita cada cliente para visualizar su región de interés. Además, determina cuáles puede reutilizar, verifica su integridad y recupera aquellos que presentan errores. Con esto buscamos evitar envíos innecesarios y controlar el uso de memoria y procesamiento.

En este documento explicamos cómo implementamos el proyecto y por qué tomamos sus principales decisiones, tomando como base nuestra propuesta de PRIB. Primero presentamos el recorrido de una solicitud del cliente al servidor y después describimos los mecanismos que intervienen. Los anexos detallan los campos y formatos de los mensajes utilizados por el protocolo.

Un recorrido antes de los detalles

Supongamos que el usuario amplía una imagen y desea ver una región de 256×128 píxeles del nivel original. Si comienza en el origen, la región requiere dos bloques de 128×128 . La imagen completa permanece en el servidor.

Paso	Qué ocurre
1. Solicitar	El navegador envía el identificador de imagen, nivel, coordenadas y tamaño de región.
2. Planificar	El servidor identifica los bloques necesarios y decide el orden y el modo de envío.
3. Transferir	Envía contenido completo, referencias o diferencias, según la caché y el crédito de ese cliente.
4. Verificar	El navegador obtiene el RGB, comprueba SHA-256 y dibuja el bloque.
5. Confirmar	ACK confirma integridad; CACHE_STATE comunica retención y CREDIT_GRANT devuelve capacidad.
6. Completar	VIEW_DONE cierra la vista cuando todos los objetivos están confirmados.

Si el usuario vuelve a la misma región y sus bloques siguen en caché, podemos usar REUSE. Si cambia de región antes de terminar, damos un número nuevo a la vista: esa generación permite distinguir las respuestas actuales de las antiguas. Cada pestaña tiene su propia sesión, aunque varios clientes exploren la misma imagen.

Qué aporta PRIB sobre el transporte

Usamos HTTP para entregar los archivos HTML, JavaScript y CSS de la página, y WebSocket para intercambiar mensajes entre el cliente y el servidor en ambas direcciones sobre TCP.

TCP se encarga de entregar los datos en orden, recuperar los paquetes perdidos y controlar la congestión. Por su parte, PRIB decide qué bloques enviar, si puede reutilizarlos o enviar sus diferencias y cuándo enviarlos según los créditos disponibles del cliente. La recuperación de PRIB actúa cuando un bloque no supera la verificación de integridad o cuando falta una base necesaria para reconstruirlo. Estos casos se resuelven a nivel del protocolo PRIB; la recuperación de paquetes perdidos en la red corresponde a TCP.

2 Preparación y arquitectura

De una imagen grande a bloques accesibles

Antes de navegar, preparamos una pirámide de resolución. El nivel 0 conserva la imagen original y cada nivel siguiente promedia grupos de hasta 2×2 píxeles, con dimensiones redondeadas hacia arriba. La pirámide termina cuando el nivel cabe en un bloque. Así se puede mostrar una vista general pequeña y solicitar detalle solo donde interesa.

La representación común es RGB8: tres bytes por píxel, en orden de filas. Un bloque interior de 128×128 ocupa 49 152 bytes; los bloques de borde pueden ser menores. Su identidad combina `imageId` con `blockId`, cuyo formato es `nivel:columna:fila`. Por ejemplo, `0:15:7` comienza en $x = 1920$ e $y = 896$ del nivel 0.

Archivo	Contenido y función
<code>image.properties</code>	Identidad, dimensiones y niveles de la imagen.
<code>level-N.pack</code>	Bloques concatenados, RAW o zlib según el ahorro en disco.
<code>level-N.idx</code>	Registros de 48 bytes con offset, longitud, códec y SHA-256.
<code>result.txt</code> / <code>INCOMPLETE</code>	Resultado de preparación / marca que impide publicar una salida incompleta.

Consultamos en el índice la ubicación del bloque que necesitamos, sin cargar toda la imagen ni todo el índice en memoria. Durante la preparación, procesamos el PNG por filas y franjas para limitar el consumo de RAM. Aunque un bloque esté comprimido dentro del archivo de bloques (`pack`), el modo FULL lo envía como bytes RGB sin comprimir. Calculamos SHA-256 sobre esos bytes RGB y, en los demás modos, sobre el contenido RGB reconstruido, de modo que la verificación de integridad sea la misma independientemente de cómo se transmita el bloque.

Distribución del trabajo

Componente	Responsabilidad
<code>PribServer</code> y <code>PribSession</code>	HTTP/WebSocket, Selector y estado independiente de cada sesión.
<code>ImageStore</code> y <code>PrepareImage</code>	Lectura indexada, preparación de bloques y niveles.
<code>CreditWindow</code> y <code>ClientCache</code>	Capacidad de recepción e inventario de caché declarado por el cliente.
<code>BlockPriority</code> y <code>SimilarityIndex</code>	Orden de envío y búsqueda acotada de bases similares.
<code>DeltaCodec</code> y <code>web/delta.js</code>	Diferencias exactas y reconstrucción compatible Java/JavaScript.
<code>web/app.js</code> y <code>cache.js</code>	Solicitudes, RGB retenido, verificación y dibujo en Canvas.

El Selector atiende las conexiones sin necesitar un hilo por cliente. Dejamos las lecturas, los hashes y DELTA a cuatro hilos de trabajo, que devuelven sus resultados al Selector. La preparación se ejecuta en otro proceso. El servidor conserva un inventario, es decir, una lista de los bloques que cada cliente declara tener; no copia toda su caché RGB.

Elegimos bloques de tamaño fijo para limitar la memoria necesaria y facilitar su búsqueda y comparación. Esto implica enviar un bloque entero aunque solo se vea una parte. Los niveles reducidos pierden detalle al promediar píxeles; FULL y DELTA conservan exactamente el RGB del nivel solicitado. Ampliar más del 100 % agranda los píxeles, sin crear detalle.

3 Reutilización y diferencias exactas

Cómo se elige el modo de envío

Llamamos base a un bloque ya reconstruido y retenido en caché; payload es la parte del mensaje que contiene los datos del bloque o sus diferencias.

Modo	Condición	Qué se transmite
REUSE	El mismo bloque sigue retenido, con igual geometría y hash.	Referencia sin payload.
REF	Otro bloque retenido tiene geometría y contenido idénticos.	Referencia a esa base.
DELTA	Una base similar permite una diferencia con ahorro suficiente.	Tramos XOR sobre la base.
FULL	No hay base válida, no conviene DELTA o se recupera el objetivo.	RGB completo.

Primero buscamos coincidencias exactas en el inventario. REUSE usa la identidad del propio objetivo; REF permite otra identidad, incluso de otra imagen. Si no existe coincidencia, se evalúan hasta cuatro bases similares. El servidor compara el tamaño completo de cada paquete DELTA con FULL y conserva la alternativa aceptable más pequeña.

Antes de enviar, comprobamos otra vez que la base siga en el inventario del cliente. Si ya no aparece, usamos FULL. El navegador también la verifica, porque la lista del servidor puede no coincidir con lo que realmente conserva el cliente.

Búsqueda de similitud

Cada bloque obtiene una firma de 15 valores: medias R, G y B del bloque y de cuatro cuadrantes, cuantizadas entre 0 y 15. SimilarityIndex agrupa por geometría y color global; consulta hasta 27 grupos vecinos, examina como máximo 64 entradas y devuelve hasta cuatro candidatos cuya distancia total entre firmas no supere 24.

Buscamos así para no comparar cada objetivo con toda la caché. Dos bloques pueden tener colores medios parecidos y muchos píxeles distintos. En ese caso, describir las diferencias puede costar tanto como enviar FULL. Por eso la firma solo propone candidatos: comprobamos los hashes y comparamos el tamaño real de ambos paquetes antes de elegir DELTA.

Cómo se calculan las diferencias

El codificador compara base y objetivo byte a byte. Guarda tramos con posición inicial, longitud y bytes XOR; los bytes no indicados se conservan de la base. El receptor copia la base y aplica los tramos. Por ejemplo, si un byte vale 30 y debe valer 31, se envía $30 \text{ XOR } 31 = 1$; al recibirlo, $30 \text{ XOR } 1$ recupera 31.

El payload incluye 8 bytes iniciales (longitud RGB y número de tramos) y 8 bytes de metadatos por tramo, además de sus diferencias. Tres bytes adyacentes modificados necesitan 19 bytes de payload, frente a 49 152 bytes de un bloque completo. Es un ejemplo ilustrativo: la cabecera PRIB también cuenta y el ahorro depende del contenido.

Usamos DELTA solo si ahorra al menos 15 % y 256 bytes frente a FULL, incluyendo prefijo y cabecera. Elegimos estos mínimos para evitar diferencias que apenas ahorren; no afirmamos que sean los mejores valores posibles. El receptor rechaza tramos inválidos, superpuestos o incompletos y bytes sobrantes. Verifica la base y el resultado con SHA-256; las bases siempre están reconstruidas.

4 Créditos prioridad y caché

Controlar cuánto puede recibir cada cliente

El cliente inicia una ventana de 256 KiB. El servidor descuenta el coste de cada bloque al encolarlo y espera cuando no alcanza el saldo. El navegador devuelve capacidad después de procesar el mensaje. Así, un cliente lento limita su propio envío sin consumir la ventana de otro.

Coste del mensaje = 4 bytes + cabecera JSON + payload.

Saldo = capacidad inicial + bytes liberados acumulados – bytes enviados acumulados.

FULL, REUSE, REF, DELTA y reintentos consumen este coste. Los controles JSON quedan fuera de la ventana de datos, aunque usan colas limitadas. CREDIT_GRANT comunica un total acumulado y un identificador creciente: repetir una concesión no duplica capacidad. El cliente agrupa devoluciones desde 16 KiB y las fuerza al cerrar/cancelar una vista, recuperarse de un fallo o reanudar.

ACK confirma integridad; CACHE_STATE informa retención; CREDIT_GRANT devuelve capacidad. Los créditos limitan los mensajes enviados que aún no se han liberado; la caché limita el RGB que conservamos para reutilizarlo. Un bloque puede seguir en caché después de devolver el crédito de su mensaje. También podemos dibujarlo y confirmarlo sin guardarlo. Así controlamos dos recursos distintos.

Qué bloques se atienden primero

La prioridad combina proporción visible V , proximidad al centro P y coste normalizado C :
 $\text{puntuación} = 4V + 2P - 3C$. V es el área visible dividida entre el área del bloque.

$P = 1/(1 + \text{distancia normalizada entre centros})$; cada eje se divide entre el máximo de 128 y la dimensión de la vista. C es el tamaño del paquete dividido entre 53 252 bytes.

Antes de preparar un bloque estimamos su coste; después usamos su tamaño real. Una referencia puede adelantarse si tiene mejor puntuación o si cabe en el crédito mientras otro bloque debe esperar. Como los candidatos son del mismo nivel, no añadimos otro peso por detalle. Elegimos los pesos como una regla práctica. Para completar la vista, el cliente debe seguir confirmando y devolviendo capacidad.

Qué se conserva en el navegador

La caché guarda hasta 16 MiB de RGB y 1024 entradas. Para decidir qué conservar, consideramos cuándo se usó el bloque, qué tan cerca está de la vista, cuántas veces se

utilizó y si puede servir para REF. Quitamos la entrada no protegida de menor utilidad. Si el bloque nuevo aporta menos, lo dibujamos sin guardarlo; no decidimos solo por antigüedad, como haría LRU.

La fórmula es $3r + 3p + 2f + a$: $r = 1/(1 + \text{antigüedad}/32)$, $f = \min(1, \text{usos}/8)$, $p = 1/(1 + \text{distancia}/128)$ para la misma imagen y nivel, y $a = \min(2, \text{entradas con igual hash}/4)$. La antigüedad se mide en pasos del contador interno, no en segundos. Estos pesos favorecen regiones recientes y reutilizables.

PUT, DROP y CLEAR actualizan el inventario del servidor usando cacheSeq consecutivo. Mientras una vista está en proceso, protegemos los bloques que podrían servir como base para no eliminarlos de la caché. Al terminar o cancelar, se libera esa protección. Si no podemos quitar entradas, dibujamos sin guardar más. Los 16 MiB solo limitan el RGB, no toda la memoria de Chrome.

5 Cancelación recuperación y concurrencia

Cuando cambia la región visible

Cada nueva vista recibe un viewId mayor. El navegador cancela la anterior con CANCEL y solicita la nueva. El servidor marca ese trabajo como cancelado, quita lo que aún está pendiente y descarta los resultados que ya no sirven. Los bytes que ya puso en la cola de salida o envió no pueden retirarse.

El servidor envía CANCELLED después de los mensajes de la vista antigua. Al recibirlo, el navegador libera sus bases protegidas. No dibuja los bloques antiguos sobre la región nueva, pero sí devuelve la capacidad de sus mensajes. Cancelar no borra los bytes enviados que aún deben liberarse. Reconectar crea otra sesión y reinicia caché y créditos; las respuestas anteriores ya no afectan al cliente.

Cuando no se puede reconstruir un bloque

Causa	Qué detecta el cliente
BASE_MISSING	No dispone de la base requerida.
CACHE_MISS	La referencia tiene geometría o metadatos incompatibles.
HASH_MISMATCH	El hash de la base o del resultado no coincide.
DELTA_FAILED	La diferencia no se puede decodificar correctamente.

RECOVER identifica el intento fallido. El servidor responde RECOVERY_ACCEPTED y reenvía solo ese objetivo como FULL, con un nuevo transferId y nuevo consumo de crédito. Se permiten hasta dos recuperaciones por objetivo y vista. HASH_MISMATCH invalida conservadoramente la caché del cliente. Un error de cabecera, almacén o límite de reintentos cierra la conexión.

Ejemplo: falta la base de un DELTA. El cliente solicita recuperación y devuelve la capacidad del mensaje fallido; recibe FULL y verifica el objetivo. La vista concluye después de su ACK. Las métricas incluyen ambos intentos, pero cuentan un solo objetivo verificado.

Mantener independientes las sesiones

El Selector rota entre clientes y limita escritura por turno. Cada sesión tiene ventana e inventario propios, un máximo de un trabajo de bloques activo y un paquete preparado en espera. Las lecturas de disco, hashes y diferencias se delegan a cuatro trabajadores con cola de 32 tareas. El catálogo permite consultas concurrentes mientras se publican imágenes.

Límite	Valor
Conexiones / imágenes del catálogo	32 / 100
Región / bloques por vista	3840 × 2160 / hasta 558
Salida por cliente / escritura por turno	1 MiB y 128 buffers / 65 536 bytes
Cola de procesamiento del navegador	32 MiB
Heap del servidor y preparador	256 MiB por proceso en los scripts

Definimos estos límites para que las colas y la memoria no crezcan sin control. La velocidad puede variar porque CPU, disco e hilos se comparten. El servidor admite hasta 32 conexiones, pero las pruebas conservadas demuestran cuatro clientes simultáneos.

6 Preparación operación y decisiones

Agregar imágenes sin reiniciar el visor

Podemos preparar un ZIP guardado en el servidor, subir un archivo desde el navegador o indicar una URL pública directa. Se prepara una imagen a la vez y no se sobrescriben carpetas existentes. Cuando termina, CATALOG_UPDATE actualiza el selector. Preparar un ZIP o una URL continúa aunque se cierre la pestaña; subir un archivo local sí requiere mantenerla conectada.

PNG admite RGB de 8 bits sin entrelazado y se procesa por filas. JPEG, GIF y BMP se limitan a 64 MiB comprimidos, 16 millones de píxeles y 32 768 por lado; GIF usa el primer fotograma. Para varios GB preferimos ZIP, evitando la subida en fragmentos de 4 KiB. La comprobación de disco reserva todos los niveles sin comprimir, índices y 256 MiB adicionales; la salida real puede ser menor.

Inicio local y entre dos equipos

El servidor requiere JDK 21, PowerShell, el proyecto y las imágenes preparadas. Desde la raíz del repositorio ejecutamos:

```
./scripts/servidor.ps1          # acceso local
./scripts/servidor.ps1 -Red     # acceso local e IPv4
```

En el servidor abrimos `http://localhost:8080`. Para conectar otra PC, iniciamos Radmin VPN en ambos equipos, los conectamos a la misma red y después arrancamos el servidor con `-Red`. El cliente abre `http://IP-RADMIN-DEL-SERVIDOR:8080`, usando la IP real del servidor. En su Firewall permitimos TCP 8080 entrante, restringido a las IPs de Radmin de los clientes. La PC cliente solo necesita navegador y Radmin VPN.

WebSocket toma automáticamente la dirección de la página; Host, Origin y CSP se validan sin IPs fijas en el código. SHA-256 utiliza Web Crypto en localhost y la biblioteca local js-sha256 al acceder por HTTP mediante una IP. El servidor no incorpora TLS ni autenticación: los hashes comprueban integridad respecto del valor recibido, pero no la identidad del emisor.

Sin Internet, demostramos el funcionamiento mediante localhost y varias pestañas, con las imágenes y los recursos preparados previamente, sin depender de Radmin ni de descargas.

Cómo observar el funcionamiento

El panel separa las métricas de la vista actual y de la sesión. El identificador abreviado permite relacionar cada cliente con los logs del servidor, donde se muestran conexiones, solicitudes, vistas completadas, esperas por créditos, recuperaciones, cancelaciones y errores. Los eventos repetidos se agrupan para evitar saturar la terminal, sin imprimir cada bloque. Las instrucciones complementarias están en README.md y docs/demostracion.md.

Los contadores FULL, REUSE, REF y DELTA incluyen transmisiones y reintentos; los bloques verificados cuentan cada objetivo una sola vez. «RGB evitado» representa el ahorro bruto de datos RGB, sin descontar las cabeceras. Los tiempos de vista y primer bloque corresponden al procesamiento y la transferencia en la aplicación, no al tiempo de ida y vuelta de la red (RTT). La espera indica falta de créditos. El promedio de uso de caché considera cuánto tiempo se mantiene cada ocupación, y la proporción resuelta con caché incluye los bloques verificados mediante REUSE, REF y DELTA.

Por qué elegimos estas soluciones

Elegimos WebSocket porque permite enviar y recibir mensajes desde el navegador sin agregar infraestructura. Usamos RGB8 para calcular hashes y diferencias sobre el mismo formato, aunque FULL viaje sin compresión. Limitamos bloques, candidatos y memoria para controlar los recursos, aun si eso deja pasar algunas oportunidades de ahorro. Los pesos y mínimos de ahorro se pueden ajustar; no los presentamos como óptimos.

A partir de la propuesta implementamos XOR_RUNS_1 y una prioridad basada en visibilidad, cercanía y coste, sin un término separado de ganancia visual. El ahorro depende de la imagen y de la caché disponible. Lo que buscamos y comprobamos es enviar regiones con contenido exacto, controlar cada cliente de forma independiente y recuperar solo los bloques que lo necesitan.

7 Verificación y demostración

Pruebas de Funcionamiento

Las pruebas abarcan la integridad de los bloques, los modos de transmisión, los créditos, la caché, la cancelación, la recuperación y la independencia entre sesiones. Incluyen verificaciones del servidor en Java y del cliente con JavaScript y Chrome.

Los comandos se ejecutan desde la raíz del repositorio:

```
./scripts/test.ps1
./scripts/demo.ps1
./scripts/demo.ps1 -Red
./scripts/evaluar.ps1
```

Las pruebas del servidor requieren JDK 21. La demostración y la evaluación también necesitan Node.js, Chrome y Playwright instalados localmente. La demostración genera su propia muestra; la evaluación utiliza las imágenes grandes preparadas. La opción -Red comprueba el acceso por localhost e IP desde un mismo equipo; no sustituye una prueba entre dos computadoras.

Escenarios de Demostración

Acción	Evidencia observable
Ampliar al nivel 0 y arrastrar	Solo se solicitan bloques de la región; Vista completa y SHA-256 correcto.
Repetir con Ir a la región	REUSE si sigue retenida; reducción de bytes enviados.
Pausar devoluciones y pedir otra región	Un cliente espera crédito; el otro sigue. Reanudar completa la vista.
Cambiar de región rápidamente	Se descarta trabajo antiguo y se conserva la última vista.
Ejecutar la demo sintética	FULL, REF, DELTA, REUSE y cuatro causas de recuperación.

El identificador de sesión permite relacionar las métricas del panel con los eventos del servidor. Para observar el agotamiento de créditos se solicita una región nueva suficientemente amplia mientras las devoluciones están pausadas.

Como REF y DELTA dependen del contenido de los bloques y de las bases disponibles, la demostración utiliza datos controlados para reproducir ambos modos y provocar los casos de recuperación sin modificar las imágenes originales.

Resultados de una ejecución registrada

En la ejecución del 5 de octubre de 2026 se registraron 109 vistas sobre ocho imágenes, con tamaños de hasta 55 GB de datos RGB, y escenarios con cuatro clientes simultáneos. Las pruebas se realizaron en Windows con Java 21, ocho procesadores lógicos y una ventana de navegador de 1440 × 1000 píxeles. La siguiente tabla presenta algunos de los escenarios evaluados:

Escenario	Vistas	p50 ms	p95 ms	Ahorro PRIB
Primera vista del origen	8	635.5	890.0	18.14 %
Origen repetido	8	166.3	192.5	98.92 %
Navegación prolongada	40	647.1	713.6	23.10 %
Cuatro clientes	16	1385.6	2164.1	19.44 %

El p50 es la mediana del tiempo de finalización de las vistas; el p95 indica el tiempo que aproximadamente el 95 % no supera. El ahorro compara bytes PRIB con un envío FULL equivalente: incluye cabeceras y reintentos, pero excluye mensajes de control y sobrecarga de WebSocket/TCP/IP. No compara formatos PNG o JPEG.

La región repetida obtuvo mayor ahorro por la reutilización de caché; ese porcentaje no se generaliza a cualquier navegación. Los resultados complementan la demostración en vivo y pueden variar según el equipo. Las pruebas con datos controlados verifican también los modos y la recuperación. No se evaluaron imágenes de 93 GB, 32 clientes simultáneos ni todos los píxeles de cada imagen.

Anexo A Mensajes y campos de control

C indica cliente y S servidor. Los controles son JSON UTF-8 con version = 1 y type. Después de HELLO se utiliza sessionId. viewId crece por vista y transferId identifica cada intento. VIEW y VIEW_UPDATE tienen la misma semántica.

Mensaje y dirección	Campos o efecto principal
HELLO · C → S	Inicia la sesión; solo se admite una vez.
IMAGE_INFO · S → C	sessionId, images, blockSize, modos, códec y límites de crédito, caché y recuperación.
VIEW / VIEW_UPDATE · C → S	viewId, imageId, level, x, y, width, height. Región positiva y contenida en el nivel.
VIEW_ACCEPTED · S → C	Confirma región, nivel y blocks requeridos.
CREDIT_INIT · C → S	capacityBytes: entre 53 252 bytes y 1 MiB. Repetir el mismo valor no reinicia deuda.
CREDIT_GRANT · C → S	grantId y releasedBytes acumulativo. Repeticiones cubiertas no duplican saldo.
CREDIT_STATUS · C ↔ S	Consulta o informa capacidad, availableBytes, outstandingBytes, releasedBytes, grantId y state.
CACHE_STATE · C → S	cacheSeq consecutivo y operation PUT/DROP/CLEAR. PUT añade identidad, geometría, hash y firma opcional.
ACK · C → S	viewId, transferId y hash del objetivo. Debe coincidir con un intento pendiente.
CANCEL / CANCELLED · C ↔ S	viewId. La respuesta incluye tareas canceladas, bytes no enviados y deuda vigente.
RECOVER · C → S	viewId, transferId y reason de un intento pendiente.

Mensaje y dirección	Campos o efecto principal
RECOVERY_ACCEPTED · S → C	viewId, transferId, blockId, reason, attempt y mode FULL.
VIEW_DONE · S → C	Objetivos, transmisiones, modos, bytes, recuperaciones y evaluación DELTA; exige todos los ACK.
ERROR / CLOSE	ERROR informa fallo fatal; CLOSE solicita cierre normal.
CATALOG_UPDATE · S → C	images con el catálogo publicado.

Preparación y estados

PREPARE_LIST y PREPARE_ENTRIES consultan ZIP y entradas; PREPARE_START recibe zip, entry y name. PREPARE_URL recibe url y name. PREPARE_UPLOAD declara size y name; CHUNK envía offset y data base64; END termina y ABORT cancela. Todos usan el prefijo PREPARE_. PREPARE_UPLOAD_ACK confirma offset; STATUS informa progreso y ERROR un fallo de preparación sin cerrar el visor.

CREDIT_STATUS distingue WAIT_INIT, WAIT_CREDIT, SENDING e IDLE. IDLE no garantiza haber recibido todos los ACK. PREPARE_STATUS distingue IDLE, RUNNING, DONE y FAILED. Las subidas usan controles y no la ventana binaria de bloques. Los contadores acumulativos se limitan a $2^{53} - 1$ para ser exactos en JavaScript.

Anexo B Formato binario y referencias

Cada bloque viaja en un mensaje WebSocket binario: 4 bytes uint32 big endian con la longitud N de cabecera, JSON UTF-8 de N bytes y payload de M bytes. N no supera 4096 y M debe coincidir con payloadLength. El máximo contabilizado por créditos es 53 252 bytes. Los controles entrantes se limitan a 8192 bytes.

Campos de cabecera	Significado
version, type	1; BLOCK_FULL, BLOCK_REF o BLOCK_DELTA.
sessionId, viewId, transferId	Sesión, generación e intento.
imageId, level, blockId	Objetivo y nivel; blockId = nivel:columna:fila.
x, y, width, height	Geometría en píxeles del nivel.
format, codec, mode	RGB8; RAW/CACHE/XOR_RUNS_1; FULL/REUSE/REF/DELTA.
payloadLength, expectedHash	Bytes de payload y SHA-256 hexadecimal del RGB objetivo.
baseImageId, baseId, baseHash	Identidad y hash de base, solo en REUSE/REF/DELTA.

REUSE/REF usan BLOCK_REF sin payload. XOR_RUNS_1 inicia con rgbLength y runCount; cada tramo contiene offset, length y length bytes XOR. Los enteros son uint32 big endian. Los tramos están ordenados y no se solapan. Se agrupan diferencias cercanas mientras el recorrido no encuentre un byte igual a más de ocho posiciones de la última diferencia. Un candidato que alcanza el tamaño RGB se descarta antes de comparar paquetes completos.

Código fuente

La implementación de PRIB, las instrucciones de ejecución y los reportes de pruebas están disponibles en el repositorio:

github.com/LesterHernandez/CC8-Proyecto2